

Data Bench

Database Performance Report

Host	34.23.203.52	Engine	MYSQL
Tier / SKU	(Google)	Memory	22.2 GB
Buffer Pool	22784 MB	Pool Instances	1
Max Connections	4030	IO Capacity	5000 / 10000 max
IO Threads	4R / 8W	Redo Log	100 MB
Flush at Commit	1	Collation	utf8mb4_0900_ai_ci
Charset	utf8mb4	Version	8.4.8-google
Preset	Standard	Date	April 5, 2026 - 11:16 PM
Baseline Ping	0.6 ms		

Executive Summary

Test	IOPS	p50 Latency	p99 Latency	Error Rate
random_read	1,534 IOPS	0.52 ms	0.82 ms	0.00%
random_write	426.3 IOPS	1.18 ms	1.43 ms	0.00%
sequential_scan	53,676 IOPS	358.87 ms	385.20 ms	0.00%
mixed_workload	1,538 IOPS	2.12 ms	5.26 ms	0.00%
bulk_write	1,380 IOPS	697.91 ms	744.53 ms	0.00%

Stress & Integrity

Test	Result	Detail	Issues
Data Integrity	PASS	200 pages verified	0 corruptions
Concurrency	817.8 TPS	1000/1000 committed	0 deadlocks
Concurrency	980.9 TPS	2000/2000 committed	0 deadlocks

Analytical Queries

Geometric Mean	0.107 s
Power Score	1,678
Queries Passed	16 / 16
Total Runtime	3.0 s
Fastest	Q16
Slowest	Q09

Network Profile

Ping p50	0.35 ms
Ping p99	0.47 ms
Ping Jitter	0.01 ms
Connection Setup p50	0.0 ms

DNS Resolution	0.0 ms
Upload	8.1 MB/s
Download	141.1 MB/s

I/O Performance Details

These tests measure the database's storage-layer throughput and latency through SQL-level operations. Each test uses a dedicated table populated with rows containing a random binary payload of the configured block size. The thread scaling sweep runs each workload at multiple concurrency levels to build a throughput-vs-concurrency curve, identifying the optimal thread count, saturation point, and Amdahl's serial fraction. All latency measurements are captured at the application layer (including network round-trip time) and reported as a full percentile distribution.

random_read

Executes SELECT payload FROM test_table WHERE id = :random_pk using randomly chosen primary keys across a scaling thread sweep. Each operation exercises the full cloud database I/O path: B-tree index seek, single-page buffer pool fetch (or storage read on cache miss), result serialization, TLS framing, and network round-trip. This is the fundamental performance primitive for cloud databases -- directly analogous to 4KB random read IOPS on block storage. The thread sweep reveals how throughput scales with concurrency and identifies the saturation point where adding connections no longer improves performance.

IOPS	1,534	ops/s
Throughput	12.0	MB/s
Total Operations	50,000	
Error Rate	0.00	%
Duration	32.6	s

LATENCY DISTRIBUTION

Minimum	0.388	ms
p50 (Median)	0.519	ms
p75	0.560	ms
p90	0.621	ms
p95	0.679	ms
p99	0.818	ms
p99.9	1.006	ms
Maximum	95.056	ms
Mean	0.548	ms
Std Dev	0.851	ms
Jitter	0.635	ms

THREAD SCALING

Peak IOPS	2,020	at 4 th...
Saturation Point	4 threads	
Optimal Threads	2 threads	
Serial Fraction	0.4224	

Threads	IOPS	p50 (ms)	p99 (ms)	Efficiency
1	822.8	0.48	0.70	100.0%
2	1,654	0.48	0.72	100.5%
4	2,020	0.85	1.61	61.4%
8	1,949	1.86	4.50	29.6%
16	1,795	1.36	8.05	13.6%

random_write

Executes UPDATE test_table SET payload = :random_data WHERE id = :random_pk under increasing thread concurrency. Each write operation involves: index seek to locate the target row, row-level lock acquisition, in-place page modification in the buffer pool, synchronous write-ahead log (WAL/redo log) flush to durable storage, and lock release. Writes are typically 2-5x slower than reads because every write must survive a crash -- the WAL flush is the serialization bottleneck all concurrent writers share. The scaling curve for writes directly reveals WAL throughput limits and lock contention behavior.

IOPS	426.3	ops/s
Throughput	3.3	MB/s
Total Operations	50,000	
Error Rate	0.00	%
Duration	117.3	s

LATENCY DISTRIBUTION

Minimum	1.022	ms
p50 (Median)	1.179	ms
p75	1.233	ms
p90	1.288	ms
p95	1.328	ms
p99	1.427	ms
p99.9	1.864	ms
Maximum	79.455	ms
Mean	1.199	ms
Std Dev	0.468	ms
Jitter	0.443	ms

THREAD SCALING

Peak IOPS	1,059	at 8 th...
Saturation Point	4 threads	
Optimal Threads	1 threads	
Serial Fraction	0.4044	

Threads	IOPS	p50 (ms)	p99 (ms)	Efficiency
1	433.0	1.15	1.36	100.0%
2	732.4	1.22	1.86	84.6%
4	1,033	1.55	2.65	59.6%
8	1,059	2.30	5.42	30.6%
16	980.3	1.75	4.11	14.2%

sequential_scan

Executes `SELECT * FROM test_table` to perform a full table scan, reading every row in physical storage order. Unlike random I/O tests (which are latency-bound by network RTT and index seeks), sequential scan is bandwidth-bound -- it measures how fast the database can stream large result sets from storage through the buffer pool, across the network, and into the client. Performance is determined by: storage sequential read throughput, buffer pool read-ahead efficiency, result set serialization speed, TCP window scaling, and TLS encryption overhead.

IOPS	53,676	ops/s
Throughput	419.3	MB/s
Total Operations	60,000	
Error Rate	0.00	%
Duration	1.1	s

LATENCY DISTRIBUTION

Minimum	332.849	ms
p50 (Median)	358.874	ms
p75	385.202	ms
p90	385.202	ms
p95	385.202	ms
p99	385.202	ms
p99.9	385.202	ms
Maximum	385.202	ms
Mean	358.975	ms
Std Dev	26.176	ms
Jitter	0.214	ms

mixed_workload

Runs a concurrent mix of 70% point reads (SELECT by PK) and 30% single-row UPDATES across multiple threads, simulating a realistic OLTP workload. This test reveals the read/write interference factor -- how much concurrent writes degrade read performance through buffer pool page invalidation, row-level lock contention, and WAL flush serialization. The ratio of mixed IOPS to pure read IOPS quantifies this interference: a ratio below 0.6 indicates significant write-path contention that may require schema optimization or tier upgrade.

IOPS	1,538	ops/s
Throughput	12.0	MB/s
Total Operations	50,000	
Error Rate	0.00	%
Duration	32.5	s

LATENCY DISTRIBUTION

Minimum	0.510	ms
p50 (Median)	2.116	ms
p75	2.815	ms
p90	3.598	ms
p95	4.130	ms
p99	5.265	ms
p99.9	7.250	ms
Maximum	190.860	ms
Mean	2.326	ms
Std Dev	2.031	ms
Jitter	1.758	ms

bulk_write

Inserts rows in multi-row INSERT batches to measure sustained write ingestion throughput. Performance is governed by: transaction log write bandwidth (the primary bottleneck), B-tree page splits (when new keys fall between existing index entries), index maintenance overhead (each additional index multiplies insert cost), auto-statistics update triggers, and checkpoint interference. This test is directly relevant to ETL pipelines, Change Data Capture (CDC) replication, event streaming ingestion, and any workload that bulk-loads data into the database.

IOPS	1,380	ops/s
Throughput	10.8	MB/s
Total Operations	20,000	
Error Rate	0.00	%
Duration	14.5	s

LATENCY DISTRIBUTION

Minimum	675.805	ms
p50 (Median)	697.911	ms
p75	712.367	ms
p90	723.000	ms
p95	744.532	ms
p99	744.532	ms
p99.9	744.532	ms
Maximum	744.532	ms
Mean	698.418	ms
Std Dev	17.655	ms
Jitter	10.653	ms

Stress & Integrity Details

These tests validate data correctness and ACID guarantees under concurrent load. Unlike the I/O tests (which measure performance), stress tests verify that the database produces correct results under contention -- that data is not silently corrupted, that transactions are properly isolated, and that concurrent modifications do not violate application invariants. Any failure in these tests is a significant finding.

integrity_stress

Writes random binary pages of configurable size (matching the I/O block size) along with their SHA-256 cryptographic hashes into the database under concurrent thread load. After all writes complete, a statistical sample of pages is read back, and the SHA-256 hash is recomputed and compared against the stored hash. This detects: silent storage-layer corruption (bit rot), network transport errors beyond what TCP/TLS checksums catch, database engine bugs in page management, and driver-level serialization issues. Any hash mismatch is a FAIL -- a critical finding indicating data corruption that should be investigated immediately.

Integrity	PASS	
Pages Written	1000	
Pages Verified	200	
Corruptions	0	
TPS	817.8	txn/s
Committed	1000 / 1000	
Deadlocks	0	
Commit p50	7.31	ms
Commit p99	14.73	ms
Commit Mean	7.54	ms

concurrent_stress

Creates a set of bank accounts with known initial balances, then executes thousands of concurrent balance-transfer transactions (debit account A, credit account B within a single BEGIN/COMMIT block). After all transactions complete, the test verifies that SUM(all_balances) equals the original total -- the monetary conservation invariant. This tests the database's core ACID guarantees under realistic contention: atomicity (partial transfers must not persist), consistency (balances must remain valid), isolation (concurrent transfers must not corrupt each other), and durability (committed transfers must survive). The test also measures: transactions per second (TPS), deadlock frequency, commit latency distribution, and the ratio of successfully committed to total attempted transactions.

TPS	980.9	txn/s
Committed	2000 / 2000	
Deadlocks	0	
Conservation	PASS	
Balance Drift	\$0.00	

Commit p50	13.81	ms
Commit p99	28.83	ms
Commit Mean	14.42	ms

Analytical Query Details

The analytical benchmark generates a normalized star-schema dataset (zones, countries, vendors, buyers, products, inventory, sale orders, order details) and executes 16 original decision-support queries covering a range of SQL patterns: multi-way joins, correlated subqueries, EXISTS/NOT EXISTS, GROUP BY with HAVING, CASE expressions, CTEs, derived tables, and ranking queries. Each query runs cold (first execution -- no plan cache, cold buffer pool) and warm (immediate re-execution -- cached plan, warm buffer pool) to measure query compilation overhead and cache effectiveness. The geometric mean fairly weights all query durations regardless of magnitude, and the power score normalizes throughput across different scale factors for cross-comparison.

SCALE FACTOR: 0.05

Query	Duration	Rows	Cold Run	Warm Run	Speedup	Status
Q01	0.360s	7.0	0.360s	0.360s	1.0x	OK
Q02	0.131s	1,233	0.131s	0.131s	1.0x	OK
Q03	0.034s	572.0	0.034s	0.034s	1.0x	OK
Q04	0.050s	5.0	0.050s	0.050s	1.0x	OK
Q05	0.064s	5.0	0.064s	0.064s	1.0x	OK
Q06	0.074s	1.0	0.074s	0.074s	1.0x	OK
Q07	0.350s	4.0	0.350s	0.350s	1.0x	OK
Q08	0.159s	2.0	0.159s	0.159s	1.0x	OK
Q09	1.115s	0.0	1.115s	1.115s	1.0x	OK
Q10	0.061s	2,099	0.061s	0.061s	1.0x	OK
Q11	0.137s	1,647	0.137s	0.137s	1.0x	OK
Q12	0.125s	2.0	0.125s	0.125s	1.0x	OK
Q13	0.077s	24.0	0.077s	0.077s	1.0x	OK
Q14	0.086s	1.0	0.086s	0.086s	1.0x	OK
Q15	0.085s	1.0	0.085s	0.085s	1.0x	OK
Q16	0.023s	1,521	0.023s	0.023s	1.0x	OK

Network Profile Details

Network profiling isolates the transport layer from the database compute layer, allowing you to distinguish whether performance is network-bound or compute-bound. Each measurement targets a specific segment of the connection lifecycle: DNS resolution (hostname-to-IP lookup), TCP+TLS connection establishment (handshake overhead your connection pool amortizes), bare ping latency (the irreducible floor for any operation -- SELECT 1 with zero payload), first-byte latency (query compilation + initial I/O + first result row), and bidirectional bandwidth (bulk INSERT upload and bulk SELECT download throughput). These measurements provide the baseline context for interpreting all other test results.

PING LATENCY (SELECT 1)

p50	0.347	ms
p75	0.354	ms
p90	0.375	ms
p95	0.382	ms
p99	0.475	ms
p99.9	0.475	ms
Mean	0.352	ms
Std Dev	0.023	ms
Jitter	0.013	ms

CONNECTION & BANDWIDTH

DNS Resolution	0.0	ms
Connection Setup p50	0.0	ms
Connection Setup p99	0.0	ms
First Byte p50	25.14	ms
First Byte p99	33.26	ms
Upload Bandwidth	8.1	MB/s
Download Bandwidth	141.1	MB/s

Appendix A -- Test Definitions

Complete reference of every test executed by Data Bench, including methodology, what is measured, and how to interpret the results.

Random Read (I/O)

Executes `SELECT payload FROM test_table WHERE id = :random_pk` using randomly chosen primary keys across a scaling thread sweep (e.g., 1, 2, 4, 8, 16, 32 threads). Each operation exercises the full cloud database I/O path: B-tree clustered index seek ($O(\log n)$ complexity), single-page buffer pool fetch (or storage-layer read on cache miss), result serialization into the wire protocol (TDS, PostgreSQL protocol, or MySQL protocol), TLS record framing and encryption, and TCP network round-trip back to the client.

This is the fundamental performance primitive for cloud databases -- directly analogous to 4KB random read IOPS on block storage. The thread sweep builds a throughput-vs-concurrency curve that reveals the saturation point (where adding connections stops improving IOPS), the optimal thread count (highest IOPS-per-connection), and Amdahl's serial fraction (the portion of the workload that cannot be parallelized). Compare measured IOPS against your cloud tier's documented limits to determine headroom.

Random Write (I/O)

Executes `UPDATE test_table SET payload = :random_data WHERE id = :random_pk` under increasing thread concurrency. Each write involves: index seek to locate the target row, row-level lock acquisition (or page lock on some engines), in-place page modification in the buffer pool, synchronous write-ahead log (WAL / redo log / transaction log) flush to durable storage, and lock release.

Writes are typically 2-5x slower than reads because every write must survive a crash -- the synchronous WAL flush is the serialization bottleneck that all concurrent writers share. The thread scaling curve for writes directly reveals: WAL throughput limits, lock contention behavior under increasing concurrency, and the point at which adding connections increases latency without improving throughput. A high serial fraction (> 0.15) indicates the WAL is the binding constraint.

Sequential Scan (I/O)

Executes `SELECT * FROM test_table` -- a full table scan reading every row in physical storage order. Unlike random I/O tests (which are latency-bound by network RTT and index seeks), sequential scan is bandwidth-bound -- it measures how fast the database can stream large result sets from storage through the buffer pool, across the network, and into the client.

Performance is determined by: storage sequential read throughput, buffer pool read-ahead efficiency (how well the engine pre-fetches pages it knows it will need), result set serialization speed, TCP window scaling behavior, and TLS encryption overhead on large payloads. The throughput (MB/s) should approach your cloud tier's documented sequential I/O bandwidth minus protocol overhead.

Mixed Read/Write (I/O)

Runs a concurrent mix of 70% point reads (`SELECT` by PK) and 30% single-row `UPDATES` across multiple threads, simulating a realistic OLTP workload. This test reveals the read/write interference factor -- how much concurrent writes degrade read performance through buffer pool page invalidation, row-level lock contention, and WAL flush serialization.

The ratio of mixed IOPS to pure read IOPS quantifies this interference: a ratio near 1.0 means writes cause minimal disruption; a ratio below 0.6 indicates significant write-path contention that may require schema optimization (e.g., covering indexes to avoid lookups), workload isolation (read replicas), or tier upgrade (faster log subsystem). This is the most representative test for typical OLTP application performance.

Bulk Insert (I/O)

Inserts rows in multi-row `INSERT` batches (e.g., 100 rows per statement) to measure sustained write ingestion throughput. Performance is governed by: transaction log write bandwidth (the primary bottleneck for most cloud databases), B-tree page splits (when new keys fall between existing index entries, the page must be split and redistributed), index maintenance overhead (each additional secondary index multiplies the per-row insert cost), auto-statistics update triggers (which can pause inserts while statistics are recomputed), and checkpoint interference (periodic dirty-page flushing competes for I/O bandwidth).

Directly relevant for ETL pipelines, Change Data Capture (CDC) replication targets, event streaming ingestion, and any workload that bulk-loads data. Results are reported as rows/second and MB/s.

Connection Pool Stress (I/O)

Fires bursts of SELECT 1 queries at escalating concurrency levels (1 â€ˆ 5 â€ˆ 10 â€ˆ 20 â€ˆ 40 â€ˆ 60 â€ˆ 80 â€ˆ 100 â€ˆ 150 â€ˆ 200 simultaneous connections) to map the connection handling behavior under stress. At low concurrency, checkout latency should match the baseline ping. As concurrency increases, the test identifies: the saturation point (where latency begins to spike due to connection pool exhaustion, database-side worker thread scheduling, or session memory pressure), the failure threshold (where new connections are refused with max_connections or login timeout errors), and per-burst p50/p99 latency.

Use these results to size your application's max_pool_size parameter and to detect cloud-imposed connection limits that may not be documented. Many cloud database tiers silently throttle connections before reaching the advertised max_connections value.

Data Integrity (Stress)

Writes random binary pages of configurable size (matching the I/O block size) along with their SHA-256 cryptographic hashes into the database under concurrent thread load. After all writes complete, a statistical sample of pages is read back, the SHA-256 hash is recomputed from the retrieved binary data, and compared against the stored hash.

This detects: silent storage-layer corruption (bit rot in the underlying cloud storage), network transport errors beyond what TCP/TLS checksums catch (such as memory corruption in network interface buffers), database engine bugs in page management (incorrect page splits, torn writes), and driver-level serialization issues (endianness problems, binary encoding errors). SHA-256 has a collision probability of approximately 1 in 2^{128} , making false negatives effectively impossible.

Any hash mismatch is a FAIL -- a critical finding indicating data corruption somewhere in the end-to-end pipeline. This should be investigated immediately and escalated to the cloud provider if the storage layer is suspected.

Concurrent Transactions (Stress)

Creates a set of bank accounts with known initial balances, then executes thousands of concurrent balance-transfer transactions (debit account A by \$X, credit account B by \$X within a single BEGIN/COMMIT block). After all transactions complete, the test verifies that SUM(all_balances) equals the original total -- the monetary conservation invariant.

This tests the database's core ACID guarantees under realistic contention:

- â€¢ Atomicity -- partial transfers (debit without credit) must not persist after a crash or rollback
- â€¢ Consistency -- account balances must remain non-negative (or at least the sum must be conserved)
- â€¢ Isolation -- concurrent transfers must not see each other's intermediate states
- â€¢ Durability -- committed transfers must survive process termination

The test also measures: transactions per second (TPS), deadlock frequency and recovery, commit latency distribution (p50/p99), and the committed/total ratio. A balance drift > \$0.01 is a FAIL indicating a serialization violation.

Isolation Level Testing (Stress)

Systematically tests each transaction isolation level supported by the database engine by executing known anomaly-producing patterns and checking whether the database correctly prevents them.

Isolation levels tested (where supported): READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE, and SNAPSHOT.

Anomalies tested:

- â€¢ Dirty Read -- Transaction A reads uncommitted data written by Transaction B. Should be prevented at READ COMMITTED and above.
- â€¢ Non-Repeatable Read -- Transaction A reads a row, Transaction B modifies and commits it, Transaction A re-reads and gets different data. Should be prevented at REPEATABLE READ and above.
- â€¢ Phantom Read -- Transaction A executes a range query, Transaction B inserts a new row in that range and commits, Transaction A re-executes and sees the new row. Should be prevented at SERIALIZABLE.
- â€¢ Write Skew -- Two transactions read overlapping data, then make disjoint updates that together violate a constraint. Only prevented by SERIALIZABLE or explicit locking.

This validates that the database's actual behavior matches its documented isolation guarantees -- some cloud databases advertise SERIALIZABLE but implement it as snapshot isolation, which does not prevent write skew.

Analytical Queries (Decision Support)

Generates a normalized star-schema dataset (zones, countries, vendors, buyers, products, inventory, sale orders, order details) scaled by a configurable factor, then executes 16 original decision-support queries covering the full range of SQL optimizer code paths:

- â€¢ Multi-way hash/merge/nested-loop joins across 4+ tables
- â€¢ Correlated subqueries with EXISTS and NOT EXISTS
- â€¢ Aggregation with GROUP BY, HAVING, CASE expressions
- â€¢ Common Table Expressions (CTEs) and derived tables
- â€¢ ORDER BY with LIMIT/TOP for ranking queries
- â€¢ LEFT JOIN distributions and NULL handling
- â€¢ Date range filtering and string pattern matching

Each query runs cold (first execution -- no plan cache, cold buffer pool) and warm (immediate re-execution -- cached plan, warm buffer pool) to measure plan compilation overhead and cache effectiveness. The cold/warm ratio reveals how much the

Network Profiling

Isolates the transport layer from the database compute layer to distinguish network-bound vs. compute-bound performance limits. Each measurement targets a specific segment of the connection lifecycle:

â€¢ DNS Resolution -- Times the hostname-to-IP resolution through the OS resolver, detecting slow resolvers, CNAME chain traversal, and DNS cache behavior.

â€¢ Ping Latency -- Executes `SELECT 1` repeatedly (30-100 pings depending on preset) with full percentile distribution (p50, p75, p90, p95, p99, p99.9). This is the irreducible floor for any database operation.

â€¢ Connection Setup -- Times TCP three-way handshake + TLS negotiation (2 RTTs for TLS 1.2, 1 RTT for TLS 1.3) + database authentication + session initialization. This is the cost your connection pool amortizes with every reused connection.

â€¢ First Byte Latency -- Time from `conn.execute()` to first row received for a 1000-row query. Measures query compilation + index scan initiation + first page read + initial network transfer. The delta from ping latency reveals server-side query processing overhead.

â€¢ Upload Bandwidth -- Client-to-database transfer rate via timed bulk `INSERT` of rows with 1KB payloads, reported in MB/s.

â€¢ Download Bandwidth -- Database-to-client transfer rate via timed `SELECT` of all bandwidth test rows, reported in MB/s.

These measurements provide the baseline context needed to interpret all other test results -- if your ping is 50ms, every operation pays at least 50ms regardless of the database tier.

Appendix B -- Metric Definitions

Complete reference of every metric reported by Data Bench. Metrics are grouped by category. Thresholds and interpretation guidance are provided for each.

Throughput Metrics

IOPS

I/O Operations Per Second -- the count of discrete SQL operations (SELECT or UPDATE of a single row by primary key) completed per wall-clock second across all active threads. Directly analogous to random-access IOPS on block storage. Bounded by: network round-trip time, connection pool depth, row-level lock contention, and the backend storage I/O subsystem. Compare against your cloud tier's documented IOPS limit to determine remaining headroom.

Throughput (MB/s)

Aggregate data transfer rate measured at the application layer. Calculated as $(\text{block_size} \times \text{completed_operations}) / \text{elapsed_time}$. Includes all overhead: SQL protocol framing, TLS encryption, TCP segmentation, and client-side deserialization. Compare with your cloud tier's documented network bandwidth per vCPU to distinguish NIC saturation from storage bottlenecks.

TPS

Committed transactions per second. Each transaction performs a complete balance transfer between two random accounts: read balance → compute new balance → update row → commit. Encompasses the full ACID cost: lock acquisition, WAL synchronous write, group commit flush latency, and lock release. TPS is typically 10-100× lower than IOPS because each transaction involves multiple round-trips and synchronous durability guarantees.

Rows/sec

Insert throughput for bulk write operations -- the number of rows successfully inserted per second during batch INSERT statements. Affected by: transaction log bandwidth, page split frequency, index maintenance cost, and auto-statistics triggers.

Error Rate

Fraction of operations that raised an exception (timeout, connection reset, deadlock abort, etc.) as a percentage of total attempted. Well-provisioned cloud databases should show < 0.1%. Rates above 1% indicate: connection pool exhaustion, DTU/vCore limit throttling, hitting max_connections, or aggressive query timeout settings.

Latency Metrics

p50 (Median)

The 50th percentile -- half of all operations complete faster than this value. Represents steady-state user experience. On cloud databases, p50 is dominated by: network RTT + query plan lookup + index seek + buffer pool page read. When p50 matches your measured ping latency, there is zero server-side queuing delay.

p75

The 75th percentile -- 25% of operations exceed this latency. The gap between p50 and p75 reveals the spread of "normal" operations before tail effects appear.

p90

The 90th percentile -- 10% of operations exceed this. At this level, occasional buffer pool misses, minor lock waits, and network path variance begin to appear.

p95

The 95th percentile -- 5% of operations exceed this. Often the threshold used in SLA definitions (e.g., "95% of queries complete within 10ms").

p99

99th percentile latency -- only 1 in 100 operations exceeds this value. Tail latency at this level is typically caused by: garbage collection pauses on the database VM, log flush waits (when the WAL write queue is full), lock escalation events, buffer pool misses forcing synchronous storage I/O, or cloud-provider noisy-neighbor CPU throttling. A p99 exceeding 10× the p50 signals serious tail-latency variance.

p99.9

The 99.9th percentile -- 1 in 1,000 operations exceeds this. Captures extreme outliers: VM live migrations, storage failovers, TCP retransmissions, and cloud-level rate limiting events. Relevant for systems with strict latency SLAs.

Min / Max

The absolute fastest and slowest operations observed during the test. Min typically represents an ideal case (warm cache, no contention, fast network path). Max captures the worst-case event (could be a VM migration, storage timeout, or deadlock retry).

Mean / StdDev

Arithmetic mean and standard deviation of all operation latencies. While percentiles are more robust for skewed distributions, mean/stddev are useful for statistical modeling and confidence interval calculation.

Jitter

Mean absolute difference between consecutive latency measurements (ms). Quantifies operational stability. Low jitter (< 1ms) indicates predictable scheduling; high jitter suggests OS context switching, VM CPU steal time, variable network path latency (common in multi-AZ), or storage I/O scheduler reordering.

Coefficient of Variation (CV)

StdDev / Mean, expressed as a ratio. A dimensionless measure of relative variability. CV < 0.3 indicates consistent performance; CV > 1.0 indicates highly variable latency that makes capacity planning unreliable.

Scalability Metrics

Peak IOPS

The highest I/O operations per second observed across all thread counts in the scaling sweep. This represents the maximum throughput the database delivered during the test. Compare against your cloud tier's documented IOPS limits to gauge how much headroom remains before hitting the storage or compute ceiling.

Saturation Point

The thread count at which throughput (IOPS) stops increasing -- adding more connections yields < 5% improvement. Beyond saturation, additional concurrency only increases latency. When saturation occurs below your application's connection pool size, the database (not the client) is the bottleneck. Check cloud dashboard metrics: CPU%, DTU%, IOPS limits, or worker thread exhaustion.

Optimal Thread Count

The thread count yielding the highest IOPS-per-thread efficiency ratio. Operating above optimal wastes connections and increases tail latency. Set your application's pool_size to optimal_threads per instance, then scale horizontally if more aggregate throughput is needed.

Amdahl's Serial Fraction (f)

Derived from the scaling sweep using Amdahl's Law: $Speedup = 1 / (f + (1-f)/N)$, solved for f. Represents the fraction of the workload forced through a single-threaded bottleneck. f = 0.01-0.05: excellent parallelism (indexed reads). f = 0.05-0.15: moderate serialization (typical writes -- WAL flush). f = 0.15-0.30: significant contention (lock escalation, page latch waits). f > 0.30: heavily serialized -- throughput will never exceed ~3x single-threaded regardless of connection count.

Thread Scaling Efficiency

Percentage of ideal linear scaling achieved at each thread count. $Efficiency = (IOPS_at_N / (IOPS_at_1 \times N)) \times 100\%$. 100% means perfect linear scaling; below 50% means more than half the potential parallelism is lost to serialization overhead.

Integrity & Correctness Metrics

Integrity PASS/FAIL

SHA-256 verification result. PASS means every sampled page's recomputed hash matched the stored hash -- zero corruption detected. FAIL means at least one page was corrupted between write and read, indicating a defect in the storage layer, network transport, or database engine. This is a binary, critical finding.

Pages Written / Verified

Total binary pages written to the database and the number subsequently sampled and verified. The verification sample provides a statistical confidence level: verifying 100 out of 1,000 pages provides ~95% confidence of detecting a 3% corruption rate.

Balance Conservation (Drift)

The difference between the expected sum of all account balances and the actual sum after concurrent transactions. Drift < \$0.01 is PASS (within floating-point rounding tolerance). Any measurable drift indicates a serializability violation -- concurrent transactions corrupted each other's writes. This is a critical correctness failure.

Committed / Total Ratio

Fraction of attempted transactions that successfully committed. The gap represents: deadlock victims (rolled back by the database's deadlock detector), lock timeout aborts, serialization failures (in strict isolation modes), and connection errors. Below 90% suggests the concurrency level exceeds the database's lock manager capacity.

Deadlock Count

Number of transactions aborted due to deadlock detection (wait-for graph cycle). Some deadlocks are expected under random high-contention access. < 1% of transactions: normal. 1-5%: hot-row contention -- consider row-level partitioning. > 5%: likely requires schema redesign, reduced concurrency, or explicit lock ordering.

Analytical Query Metrics

Geometric Mean (seconds)

$\exp(\frac{1}{N} \sum \ln(\text{duration}_i)) / N$ across all successful queries. Preferred over arithmetic mean because it fairly weights queries across different magnitude ranges -- a 0.1s query and a 100s query contribute equally. This is the standard composite metric used in decision-support benchmarks. Lower is better.

Power Score

$(3600 \times \text{Scale_Factor}) / \text{Geometric_Mean}$. Represents theoretical query-streams per hour, normalized by dataset size. Higher is better. Enables cross-comparison between runs at different scale factors and different database engines. For context: power score 100 at SF=0.01 means the system could theoretically process 100 complete 16-query streams per hour.

Cold Duration

Query execution time on first run -- no plan cache, cold buffer pool. Includes: SQL parsing, query optimization (join ordering, index selection, cost estimation), plan compilation, physical execution, and result transfer. The optimizer's work is a significant fraction of cold duration for complex multi-join queries.

Warm Duration

Query execution time on immediate re-execution -- cached plan, warm buffer pool. The difference between cold and warm reveals plan cache effectiveness. A warm/cold ratio < 0.5 indicates substantial compilation overhead that benefits from plan caching. A ratio near 1.0 means the query is execution-bound, not compilation-bound.

Query Status

Per-query completion status: "ok" (completed successfully), "error" (failed with an exception -- typically syntax incompatibility, timeout, or insufficient tempdb), or "skipped" (excluded from the current preset level).

Row Count

Number of rows returned by each query. Useful for verifying correctness across database engines -- the same query at the same scale factor should return the same row count regardless of the underlying engine.

Network Metrics

DNS Resolution (ms)

Time to resolve the database FQDN to an IP address via the OS resolver. Includes local cache lookup or recursive DNS query. Cloud database hostnames often traverse CNAME chains (e.g., Azure Traffic Manager 'regional endpoint' instance). Values > 50ms suggest DNS cache misses or slow recursive resolvers.

Ping p50 / p99 (ms)

SELECT 1 round-trip time at median and 99th percentile. The irreducible latency floor for any database operation. Includes: client socket write ' network transit ' TLS record processing ' SQL parse/execute ' result serialization ' network return ' client read. Same-region: 1-5ms typical. Cross-region: 20-80ms. Every database operation pays at least this cost.

Ping Jitter (ms)

Mean absolute difference between consecutive ping samples. Quantifies network path stability. < 0.5ms: excellent (dedicated/premium networking). 0.5-2ms: typical shared cloud networking. > 5ms: routing instability or traffic shaping. High jitter degrades connection pool efficiency and complicates timeout tuning.

Connection Setup (ms)

Time to establish a brand-new database connection from scratch: TCP three-way handshake + TLS negotiation (2 RTTs for TLS 1.2, 1 RTT for TLS 1.3) + database authentication + session initialization. Typically 3-10x the ping latency. This is the cost your connection pool amortizes -- if pool_size is too small and connections churn, this overhead hits every request.

First Byte p50 (ms)

Median time from conn.execute() to first row received for a multi-row query. Unlike ping (zero-data payload), this returns 1,000 rows, measuring: query plan compilation/cache lookup + index scan initiation + first page read + result serialization + initial TCP segment. The delta from ping latency isolates server-side processing overhead.

Upload Bandwidth (MB/s)

Client-to-database transfer rate via timed bulk INSERT of 1KB-row payloads. Factors in: parameterized query parsing, transaction log writes, index maintenance, and TCP upload throughput. Compare with your cloud tier's documented ingress bandwidth limits.

Download Bandwidth (MB/s)

Database-to-client transfer rate via timed SELECT of all bandwidth test rows. Factors in: sequential scan speed, row serialization to wire protocol, TLS framing, and TCP download throughput. Cloud providers typically offer asymmetric bandwidth -- download is often higher than upload.